Team Members

1. Tapan Shashikant Patil – patil.tap@northeastern.edu
2. Gaurang Ghadi – ghadi.g@northeastern.edu

1. Problem Description

a. What problem are you solving?
   - We will implement Deep-Q-Network (DQN) from scratch for adaptive traffic signal control and explore extensions for prioritizing specific vehicle types (such as public transit) while maintaining efficient overall traffic flow.

b. Describe the problem formally, what are states, actions, and rewards?
   - We will formulate the traffic signal control as MDP with following states action and rewards:
   - State (S): At each decision step the state includes
     - Current traffic signal phase
     - Minimum green time elapsed indicator
     - Lane densities
     - Queue lengths
   - Actions (A): Discrete action space where each action selects next green phase
   - Rewards (R): Change in cumulative waiting time across all vehicle types
     Extended reward: Weighted formulation that differentially weighs waiting time based on vehicle type and occupancy.

c. What simulation domain are you using?
   - We will be using SUMO version 1.21.0 which has a RL interface SUMO-RL library version 1.4.5

d. Why is it interesting?
   - Traffic signal control is a real-world problem impacting urban mobility. It features high-dimensional continuous states, multi-objective tradeoffs, and established RL benchmarks for comparison.

e. Have other researchers already used it?
   - Yes, it's been used extensively. Current examples include DQN implementation using stable-baselines3

2. Algorithms

a. What algorithms do you use?
   Vanilla DQN (Mnih et al., 2015) - One team member implements from scratch: -
   - Q-Network: 2 hidden layers (64 units each), ReLU activation
   - Target Network: Updated every 10 episodes for stability

    - Experience Replay: 5,000 capacity, batch size 32
    - Epsilon-greedy exploration: 1.0 to 0.01, decay 0.995
    - Adam optimizer, learning rate 0.00025, discount factor 0.99

  Double DQN (Van Hasselt et al., 2016) - Other team member implements:
    - Fixes Q-value overestimation bias
    - Uses main network for action selection, target network for evaluation
    - Builds on vanilla DQN with modified target computation

  Both collaborate on priority extension (vehicle type detection, weighted reward functions)

b. What baseline methods will you compare to?
   1. Stable-Baselines3 DQN (verify our implementations)
   2. Fixed-time control (30 second green per phase)
   3. No priority baseline (all vehicles treated equally)
c. Why are these algorithms appropriate?
   - DQN is proven for traffic control in multiple papers as discrete actions fit phase selection
d. How are these algorithms typically used, and how are you using them?
   - DQN is typically used for traffic control in single-agent settings (isolated intersections) or multi-agent settings (networked intersections), with neural networks of varying sizes and custom reward functions for domain-specific objectives. We will implement DQN for traffic signal control following standard practices from the literature, with specific architectural and hyperparameter choices to be finalized during implementation.
e. Have other people used similar algorithms to solve your problems before?
   - Yes: Wei et al. (2018) Intelli Light System with DQN, long et al. (2022) multi-agent DQN for vehicle priority achieving 32% improvement, SUMO-RL repository includes DQN examples.

3. Results

a. What results do you expect to show?
   - We expect to demonstrate that our DQN implementation learns effective policies and converges during training. The performance will be compared against established baselines to verify correctness and shoe improvements from algorithmic enhancements.
b. What comparison will you do?
   - Our implementation Vs stable-baseline3
   - Algorithm variant comparison
   - Priority mechanism effectiveness evaluation
c. Are there risks for not getting all the results? If so, what will you do about it?
   - Implementation and training challenges may arise. We will conduct a mid-project assessment (around mid-March) to identify issues early. If any major difficulties occur, we will have fallback options including using existing implementations for portions of the project or focusing on the subset of planned experiments.